DAY 14 – SQL FUNDAMENTALS

Detailed Answers, Practical SQL Queries and Sample Database

Learning objectives: Understand SQL, create related database tables, use constraints and keys, insert student and course data, retrieve and filter records, update information, and delete selected records.

1. Introduction to SQL

SQL (Structured Query Language) is a standard language used to communicate with relational database management systems (RDBMS). It allows users and applications to define database structures, store information, retrieve records, and maintain data.

A relational database organizes information into tables. A table contains columns (fields) and rows (records). For example, a college database may have a Students table and a Courses table. Each student is identified by a unique student ID, and each course is identified by a course ID.

Common database systems: MySQL, PostgreSQL, SQLite, Oracle Database, and Microsoft SQL Server.

Benefits of SQL: It provides a structured way to manage large amounts of information, retrieve only required records, maintain relationships between tables, and reduce inconsistent data.

2. CREATE, ALTER, and DROP

These are commonly grouped as Data Definition Language (DDL) commands. They define or modify database objects such as databases and tables.

2.1 CREATE

CREATE makes a new database or table. The example below creates two tables for a simple college database.

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
student_name VARCHAR(60) NOT NULL,
email VARCHAR(100) UNIQUE,
age INT CHECK (age >= 16),
course_id INT
);

CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(80) NOT NULL,
duration_months INT CHECK (duration_months > 0)
);
```

The Students table stores student details. The Courses table stores course information. The constraints in this example are explained in Section 8.

2.2 ALTER

ALTER TABLE changes the structure of an existing table, for example by adding a new column.

```
ALTER TABLE Students
ADD city VARCHAR(50);
```

This adds a city column to Students. To remove a column, many database systems support:

```
ALTER TABLE Students
DROP COLUMN city;
```

Column-modification syntax can vary between database systems. Always check the syntax for the DBMS being used.

2.3 DROP

DROP removes a database object. Dropping a table removes its structure and its stored data.

```
DROP TABLE Students;
```

Warning: DROP is destructive. Use it only when the table is no longer required and after confirming that important data is backed up.

3. INSERT, UPDATE, and DELETE

These commands are used to manipulate the records stored in tables.

3.1 INSERT

INSERT INTO adds new rows. Specifying the column names makes the statement clearer and safer.

```
INSERT INTO Courses (course_id, course_name, duration_months)
VALUES (101, 'Python Programming', 3);

INSERT INTO Courses (course_id, course_name, duration_months)
VALUES (102, 'Web Development', 4);

INSERT INTO Students (student_id, student_name, email, age, course_id)
VALUES (1, 'Aarav Sharma', 'aarav@example.com', 20, 101);

INSERT INTO Students (student_id, student_name, email, age, course_id)
VALUES (2, 'Meera Patil', 'meera@example.com', 21, 102);
```

Each INSERT adds one record. Text values are written inside single quotes. The course records are inserted first because Students.course_id refers to Courses.course_id.

3.2 UPDATE

UPDATE changes values in existing records. WHERE identifies which record or records should change.

```
UPDATE Students
SET email = 'aarav.sharma@example.com', age = 21
WHERE student_id = 1;
```

This updates the email and age only for the student whose ID is 1. Without WHERE, every row in the table could be updated.

3.3 DELETE

DELETE removes selected rows while keeping the table itself.

```
DELETE FROM Students
WHERE student_id = 2;
```

This deletes the student with ID 2. If a foreign key from another table refers to this student, the database may prevent the deletion unless the relationship is handled appropriately.

Safe practice: Before running UPDATE or DELETE, run a SELECT statement with the same WHERE condition to confirm which records will be affected.

4. SELECT

SELECT retrieves information from a table.

4.1 Retrieve all columns

```
SELECT *
FROM Students;
```

The asterisk (*) means all columns.

4.2 Retrieve selected columns

```
SELECT student_name, email
FROM Students;
```

This displays only student names and email addresses.

4.3 Retrieve course records

```
SELECT course_id, course_name
FROM Courses;
```

5. WHERE, ORDER BY, and LIMIT

5.1 WHERE

WHERE filters records according to a condition.

```
SELECT *
FROM Students
WHERE age >= 20;
```

Only students aged 20 or above are returned.

5.2 ORDER BY

ORDER BY sorts results. ASC sorts from low to high or A to Z; DESC sorts from high to low or Z to A.

```
SELECT *
FROM Students
ORDER BY student_name ASC;

SELECT *
FROM Students
ORDER BY age DESC;
```

5.3 LIMIT

LIMIT restricts the number of returned rows. It is supported by MySQL, PostgreSQL, and SQLite; other systems may use TOP or FETCH FIRST.

```
SELECT *
FROM Students
ORDER BY student_id
LIMIT 5;
```

6. Filtering Records

Filtering is the process of selecting only records that satisfy one or more conditions.

6.1 Comparison operators

Operator	Meaning	Example
=	Equal to	age = 20
<>	Not equal to	age <> 20
>	Greater than	age > 18
<	Less than	age < 25
>=	Greater than or equal to	age >= 18
<=	Less than or equal to	age <= 25

6.2 AND and OR

AND requires all conditions to be true. OR returns a row when at least one condition is true.

```
SELECT *
FROM Students
WHERE age >= 18 AND course_id = 101;

SELECT *
FROM Students
WHERE age = 20 OR age = 21;
```

6.3 BETWEEN

BETWEEN checks an inclusive range, including both boundary values.

```
SELECT *
FROM Students
WHERE age BETWEEN 18 AND 25;
```

6.4 IN

IN checks whether a value matches any value in a specified list.

```
SELECT *
FROM Students
WHERE course_id IN (101, 102);
```

6.5 LIKE

LIKE searches text using wildcards. Percent (%) means any sequence of characters; underscore (_) means exactly one character.

```
-- Names beginning with M
SELECT * FROM Students
WHERE student_name LIKE 'M%';

-- Names containing 'ar'
SELECT * FROM Students
WHERE student_name LIKE '%ar%';
```

6.6 IS NULL

Use IS NULL to find missing values and IS NOT NULL to find values that are present. Do not use = NULL.

```
SELECT *
FROM Students
WHERE email IS NULL;

SELECT *
FROM Students
WHERE email IS NOT NULL;
```

7. Primary and Foreign Keys

7.1 Primary key

A **primary key** uniquely identifies each row in a table. Its values must be unique and cannot be NULL. A table has one primary-key constraint, which may consist of one column or multiple columns.

```
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(80) NOT NULL
);
```

Here, course_id uniquely identifies each course. Two courses cannot have the same course_id.

7.2 Foreign key

A **foreign key** is a column (or group of columns) in one table that refers to a primary key or other eligible unique key in another table. It helps maintain referential integrity: a student cannot refer to a course ID that does not exist in the Courses table.

```
CREATE TABLE Students (
student_id INT PRIMARY KEY,
student_name VARCHAR(60) NOT NULL,
course_id INT,
FOREIGN KEY (course_id)
REFERENCES Courses(course_id)
);
```

In this example, Students.course_id is the foreign key and Courses.course_id is the referenced primary key. A foreign key may be NULL unless the column is also declared NOT NULL.

7.3 Difference between primary and foreign keys

Primary key	Foreign key

Uniquely identifies a row in its own table.	Creates a relationship to a key in another (or the same) table.
Must be unique and not NULL.	Values can repeat; NULL may be allowed unless restricted.
A table has one primary-key constraint.	A table can have multiple foreign-key constraints.

8. SQL Constraints

Constraints are rules applied to table columns to help keep data accurate and valid.

• **NOT NULL:** Prevents a column from storing NULL values.

• **UNIQUE:** Prevents duplicate values in the constrained column or column combination.

• **PRIMARY KEY:** Uniquely identifies each row; combines uniqueness and non-NULL requirements.

• **FOREIGN KEY:** Enforces a valid relationship with a referenced key.

• **CHECK:** Requires values to satisfy a condition, such as age >= 16.

• **DEFAULT:** Supplies a value when an INSERT does not provide one for that column.

```
CREATE TABLE ExampleStudents (
student_id INT PRIMARY KEY,
student_name VARCHAR(60) NOT NULL,
email VARCHAR(100) UNIQUE,
age INT CHECK (age >= 16),
status VARCHAR(20) DEFAULT 'Active'
);
```

Constraints can be declared while creating a table or, where supported, added later using ALTER TABLE. Exact support and syntax can vary by database system.

9. Complete Practical Task – Student and Course Database

The following script creates the tables, inserts sample course and student records, retrieves and filters data, updates a student, and deletes a selected record. Run it in order. The SQL syntax is suitable for common systems such as MySQL and PostgreSQL, with minor variations possible in other systems.

Step 1: Create tables

```
CREATE TABLE Courses (
course_id INT PRIMARY KEY,
course_name VARCHAR(80) NOT NULL UNIQUE,
duration_months INT CHECK (duration_months > 0)
);

CREATE TABLE Students (
student_id INT PRIMARY KEY,
student_name VARCHAR(60) NOT NULL,
email VARCHAR(100) UNIQUE,
age INT CHECK (age >= 16),
course_id INT,
status VARCHAR(20) DEFAULT 'Active',
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Step 2: Insert course records

```
INSERT INTO Courses (course_id, course_name, duration_months)
VALUES
(101, 'Python Programming', 3),
(102, 'Web Development', 4),
(103, 'SQL Fundamentals', 2);
```

Step 3: Insert student records

```
INSERT INTO Students
(student_id, student_name, email, age, course_id, status)
VALUES
(1, 'Aarav Sharma', 'aarav@example.com', 20, 101, 'Active'),
```

```
(2, 'Meera Patil', 'meera@example.com', 21, 102, 'Active'),
(3, 'Rohan Deshmukh', 'rohan@example.com', 19, 103, 'Active'),
(4, 'Sara Khan', 'sara@example.com', 22, 101, 'Active'),
(5, 'Isha Joshi', 'isha@example.com', 20, 102, 'Pending');
```

Step 4: Retrieve records

```
-- Display all students
SELECT * FROM Students;

-- Display all courses
SELECT * FROM Courses;

-- Display only student names and ages
SELECT student_name, age FROM Students;
```

Step 5: Filter and sort records

```
-- Students aged 20 or above
SELECT * FROM Students
WHERE age >= 20;

-- Students enrolled in course 101
SELECT * FROM Students
WHERE course_id = 101;

-- Students from courses 101 or 102
SELECT * FROM Students
WHERE course_id IN (101, 102);

-- Sort students by age, highest first
SELECT * FROM Students
ORDER BY age DESC;

-- Display the first three students by ID
SELECT * FROM Students
ORDER BY student_id
LIMIT 3;
```

Step 6: Retrieve students with course names

A JOIN combines related information from two tables. This query uses the foreign-key relationship to display each student's course name.

```
SELECT
Students.student_id,
Students.student_name,
Courses.course_name
FROM Students
INNER JOIN Courses
ON Students.course_id = Courses.course_id;
```

Step 7: Update student information

```
UPDATE Students
SET email = 'aarav.sharma@example.com',
age = 21
WHERE student_id = 1;

-- Verify the update
SELECT * FROM Students
WHERE student_id = 1;
```

Step 8: Delete a selected student

```
DELETE FROM Students
WHERE student_id = 5;

-- Verify that student 5 is no longer present
SELECT * FROM Students
WHERE student_id = 5;
```

The deletion targets only student_id 5. The Students table remains available. If another table references this student, foreign-key rules may restrict deletion or require an appropriate relationship action.

10. Expected Output

After running the creation and insertion statements successfully, the database should contain two tables: Courses and Students. Courses should contain three sample course records, and Students should contain five sample student records before the example DELETE is executed.

The SELECT queries should display the requested records. Filtering should return only matching students; ORDER BY should sort the result; LIMIT should restrict the number of rows. The UPDATE should change the selected student's email and age. The DELETE should remove only the selected student record.

Expected outcome: Database tables created; student and course records inserted; SQL queries practiced successfully; records retrieved and filtered; student information updated; and a selected record deleted.

11. Common Mistakes and Best Practices

• Create the referenced Courses table and insert its course records before inserting Students that refer to those courses.

• Use matching data types for foreign-key and referenced-key columns.

• Use WHERE with UPDATE and DELETE unless changing or deleting every row is intentional.

• Use IS NULL rather than = NULL.

• Use ORDER BY when you need a predictable order; SQL does not guarantee row order without it.

• Check the rows with SELECT before destructive operations.

• Use backups and transactions for important production data.

• Do not assume every SQL database uses identical syntax for LIMIT, ALTER TABLE, or other database-specific features.

12. Conclusion

SQL is an essential skill for storing, managing, and retrieving relational data. DDL commands such as CREATE, ALTER, and DROP manage database structures. INSERT, UPDATE, and DELETE manage records, while SELECT retrieves data. WHERE, ORDER BY, LIMIT, and filtering operators make queries more useful. Constraints, primary keys, and foreign keys help maintain accuracy and relationships between tables. By completing the practical student-and-course database task, learners gain a foundation for using SQL in real applications.